AQI Predictor: End-to-End Serverless Forecasting for Karachi

Maha Asim

June 7, 2026

1 Introduction

The Pearls AQI Predictor is a serverless machine learning system that forecasts the Air Quality Index for Karachi, Pakistan over the next three days. All data is sourced from the Open-Meteo API, which provides hourly pollutant and weather readings at no cost. Features are stored in MongoDB Atlas and predictions are served through a Flask backend and Streamlit dashboard, both deployed to the cloud.

A key design decision was to train the model to predict AQI one hour ahead and then call it 72 times in a loop to produce the full three-day forecast. At each step, weather values are pulled from the Open-Meteo 3-day forecast and lag features are updated using the predictions already made. This ensures the forecast actually changes across days rather than repeating the same value.

2 System Architecture

The system is broken into four parts. The first is data ingestion. A backfill script seeds 90 days of historical data using the Open-Meteo archive endpoint, and a feature pipeline script runs every hour via GitHub Actions to add the latest record, while the praining pipeline runs daily. The second is the feature store, which is a MongoDB Atlas collection that holds one document per hour containing all raw and engineered features. The third is the model registry, also on MongoDB Atlas, where trained model files are stored using GridFS alongside their metrics and a flag marking which one is currently best. The fourth is the serving layer: a Flask API that loads the best model at startup and a Streamlit dashboard showing the current AQI, 3-day forecast, pollutant levels, feature importance from SHAP, and 7-day history.

3 Data and Exploratory Data Analysis

The dataset has 2,467 hourly records from 22 February to 5 June 2026. AQI values go from 32 to 161 with a mean of 80.2. About 82% of hours fall in the Moderate category.

The most important finding from EDA was how strongly the previous hour's AQI predicts the next hour's AQI. The Pearson correlation between `aqi_lag_1h` and the target is 0.97, which is unusually high and means recent history is the most valuable signal by a large

margin. Among the pollutants, $PM_{2.5}$ ($r = 0.65$) and PM_{10} ($r = 0.51$) had the strongest relationship with AQI. Weather features like temperature and humidity had weak linear correlations below 0.15, though they contribute through interaction terms.

Looking at time patterns, February had the highest average AQI at 87.4 and June the lowest at 74.9. By hour, AQI peaks around 18:00 and is lowest at midnight. When we looked at the model's errors by hour, there was a clear pattern of under-prediction during evening hours and over-prediction late at night, which we addressed in feature engineering.

A data quality problem was also found during EDA. Around 75% of weather values in the database were zero. This happened because the backfill was using the wrong Open-Meteo endpoint. The forecast endpoint only has data for recent and future dates and returns null for historical hours. The code was using `value or 0` which turned those nulls into zeros without any warning. Switching to the archive endpoint and fixing the null handling resolved the issue, and all affected records were overwritten.

4 Feature Engineering

36 features were built in total. The raw pollutants $PM_{2.5}$, PM_{10}, NO_2, O_3, CO, and SO_2 were included directly. Log versions of the skewed ones were added since their distributions were heavily right-skewed. Weather features temperature, humidity, pressure, and wind speed were included after fixing the zero-value issue. Two wind interaction features were created by dividing $PM_{2.5}$ and PM_{10} by wind speed, since higher wind disperses pollution and EDA confirmed these had useful correlation with AQI.

For AQI history, lag values were created at 1, 2, 3, 6, 12, 24, and 48 hours back, along with rolling averages at 3, 6, 12, and 24 hour windows. A 3-hour and 6-hour momentum feature was also added to capture whether AQI is rising or falling. A rolling standard deviation over 6 hours was included to tell the model whether AQI is currently stable or volatile.

For time, hour and month were encoded as sin and cos values rather than raw numbers, since hour 23 and hour 0 are adjacent in reality but would appear far apart as integers. A binary flag called `peak_traffic_hour` was set to 1 for hours 15 to 20 after residual analysis showed the model was consistently under-predicting during evening rush hour.

Three features were removed after EDA showed they added no value. The humidity interaction terms had near-zero correlation because humidity was mostly zeros during the initial backfill. Day of week had $r = 0.03$ and AQI change rate had $r = 0.12$, both too weak to be useful.

5 Model Training and Evaluation

Five models were trained: Ridge, Random Forest, Extra Trees, XGBoost, and Gradient Boosting, plus a stacking ensemble that combined the three tree models. For hyperparameter tuning, TimeSeriesSplit cross-validation with 5 folds was used instead of regular cross-validation. This matters because with time series data, regular cross-validation can accidentally use future data to train, which gives unrealistically good scores. TimeSeriesSplit always trains on earlier data and tests on later data.

The final holdout evaluation used the last 20% of records chronologically, covering 14 May to 3 June 2026.

Table 1: Model Results

Model	Holdout			TimeSeriesCV		
	RMSE	MAE	R^2	RMSE	MAE	R^2
Ridge	12.99	2.87	0.404	—	—	—
Random Forest	8.30	2.03	0.757	7.28	3.45	0.832
Extra Trees	8.09	1.99	0.769	7.19	3.46	0.838
XGBoost (winner)	8.64	2.21	0.737	6.06	2.84	0.876
Gradient Boosting	8.44	2.08	0.749	6.35	3.03	0.868
Stacking	8.70	2.45	0.733	6.87	3.92	0.849

XGBoost was selected as the best model with a TimeSeriesCV R^2 of 0.876 and RMSE of 6.06. The holdout R^2 is lower at 0.737 because that specific test window happened to be an unusual period for Karachi. AQI dropped to an average of 59.6 in the week of 17 May and then jumped to 87.9 the following week, a 28-point swing caused by pre-monsoon sea breeze conditions that were not represented in the training data. The cross-validation score is more reliable because it averages results across five different time windows rather than depending on one.

6 SHAP Interpretability Analysis

SHAP was used to understand which features drive the model's predictions. TreeExplainer was run on 500 recent observations.

The results matched what EDA suggested. `aqi_lag_1h` had by far the highest importance with a mean absolute SHAP value of 9.51, more than eight times the next feature. The top ten features were `aqi_lag_1h` (9.51), `aqi_rolling_12h` (1.17), `aqi_lag_6h` (1.02), `aqi_momentum_3h` (0.99), `aqi_lag_3h` (0.89), `pm2_5` (0.73), `aqi_lag_2h` (0.65), `aqi_std_6h` (0.55), `aqi_rolling_6h` (0.44), and `o3` (0.38).

Grouping the features, lag and rolling history accounts for 73.1% of predictive power, time features 12.0%, pollutants 9.4%, and weather 5.5%. $PM_{2.5}$ ranking sixth confirms it adds something beyond what the lag features already capture. The volatility feature `aqi_std_6h` ranking eighth shows that knowing whether AQI is currently stable or fluctuating is useful information for the model.

7 Challenges and How They Were Overcome

The first issue was that the forecast was returning the same AQI for all 72 hours. The loop was only updating the hour and date at each step while everything else stayed the same. The fix was to fetch the Open-Meteo 3-day forecast at prediction time and update weather and pollutant values at each step, with lag features recomputed from the predictions already made.

The second issue was the weather data being zero for 75% of records, which was only caught during EDA. As described above, this was caused by using the wrong API endpoint and a Python null handling bug. After the fix the mean temperature in the data went to 27.95 degrees and humidity to 70.74%, which are realistic values for Karachi.

The third issue was the model scoring worse than simply predicting that AQI stays the same as the previous hour. That naive approach gets an RMSE of 4.35 while the trained model was getting 8.31. Residual analysis showed the model was missing a consistent pattern during evening hours. Adding the `peak_traffic_hour` flag, `aqi_std_6h`, and `aqi_lag_48h`, and switching hyperparameter search to TimeSeriesCV brought the cross-validation RMSE down to 6.06.

The fourth issue was XGBoost crashing inside the stacking ensemble due to a version conflict with scikit-learn. XGBoost was removed from the stack and replaced with Extra Trees. It was kept as a standalone model and ended up being selected as the winner.

8 Conclusion

The system works end to end. It collects data automatically every hour, stores features in the cloud, trains and evaluates models with proper time-series methodology, and serves a live 72-hour forecast that changes meaningfully across all three days. The best model achieves a TimeSeriesCV R^2 of 0.876 and RMSE of 6.06 AQI points. The lower holdout score reflects one unusual test window rather than a general weakness. As more data is collected across Karachi's monsoon and winter seasons, the model will be retrained and performance should improve further.